



MODULE 6 — RÉFÉRENCE COMPLÈTE

Les Expressions Régulières en PHP

Définition · Syntaxe · Métacaractères · Fonctions preg_*
Tous les détails expliqués avec des exemples concrets

Guide complet — Sonatel Academy
Préparé pour Khadija — Dakar, Sénégal

1. Définition et utilité des expressions régulières

DÉFINITION

Une **expression régulière** (aussi appelée *regex* ou *regexp*, de l'anglais *Regular Expression*) est une **séquence de caractères qui décrit un motif (pattern) de texte**. On l'utilise pour chercher, valider, extraire ou remplacer du texte selon des règles précises et complexes.

ANALOGIE SIMPLE

Une regex, c'est comme un **tamis intelligent**. Si tu verses du texte dedans, il retient uniquement les parties qui correspondent à la forme du tamis. Par exemple, un tamis en forme "d'email valide" ne laissera passer que les adresses email correctement formées, et rejettera tout le reste.

À quoi servent les expressions régulières ?

- **Validation** : vérifier qu'un email, un numéro de téléphone ou un code postal a le bon format.
- **Recherche** : trouver toutes les occurrences d'un motif dans un texte (tous les mots en majuscules, tous les liens URL...).
- **Extraction** : récupérer des parties spécifiques d'un texte (le nom de domaine d'un email, les chiffres d'une chaîne...).
- **Remplacement** : remplacer du texte qui correspond à un motif par autre chose.
- **Découpage** : couper une chaîne selon un séparateur complexe.

Historique et standards : POSIX vs PCRE

Standard	Signification	En PHP
POSIX	Portable Operating System Interface — ancien standard Unix pour les regex	Fonctions <code>ereg_*</code> — supprimées depuis PHP 7 !
PCRE	Perl Compatible Regular Expressions — standard moderne, plus puissant	Fonctions <code>preg_*</code> — à utiliser impérativement

IMPORTANT

En PHP moderne (7+), on utilise **uniquement les fonctions PCRE** (`preg_match`, `preg_replace`...). Les anciennes fonctions POSIX (`ereg_match`, `ereg_replace`...) ont été complètement supprimées. Ne jamais les utiliser.

Premier exemple simple : valider un email

```
<?php
$email = "khadija@sonatel.sn";

// Le motif regex pour un email simple
$pattern = '/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/';

if (preg_match($pattern, $email)) {
    echo "Email valide !";      // s'affiche ici
} else {
    echo "Email invalide !";
}

?>
```

ON DÉCHIFFRE CE PATTERN ENSEMBLE

/ délimiteurs · ^ début de chaîne · [a-zA-Z0-9._%+-]+ un ou plusieurs caractères alphanumériques ou spéciaux · @ le symbole @ · [a-zA-Z0-9.-]+ le nom de domaine · \. un point · [a-zA-Z]{2,} l'extension (min 2 lettres) · \$ fin de chaîne. Tout cela sera expliqué en détail dans les pages suivantes !

2. Les délimiteurs et les caractères littéraux

Les délimiteurs en PHP

DÉFINITION

En PHP, toute expression régulière doit être encadrée par des **délimiteurs** : des caractères qui marquent le début et la fin du motif. Le délimiteur le plus utilisé est le slash `/`, mais on peut utiliser d'autres caractères comme `#`, `~`, `!` ou même des accolades `{}`.

Délimiteur	Exemple	Quand l'utiliser
<code>/</code>	<code>/bonjour/</code>	Le plus courant. À éviter si le motif contient des slash.
<code>#</code>	<code>#https://exemple\.com#</code>	Utile quand le motif contient des slash (URLs).
<code>~</code>	<code>~\d{4}~</code>	Alternative propre et lisible.
<code>{}</code>	<code>{\w+}</code>	Style accolades, moins fréquent.

```
<?php
// Ces 3 regex font exactement la même chose
$p1 = '/bonjour/';
$p2 = '#bonjour#';
$p3 = '~bonjour~';

// Pour une URL, utiliser # évite d'échapper les /
$url      = '#https://www\.google\.com#'; // propre
$url_slash = '/https:\\/\\/www\.google\.com/'; // les / doivent être échappés

?>
```

Les caractères littéraux

DÉFINITION

Un **caractère littéral** est un caractère ordinaire dans une regex : il correspond exactement à lui-même dans le texte. La lettre `a` correspond à la lettre "a", le chiffre `3` correspond au chiffre "3", etc. La plupart des lettres et chiffres sont des caractères littéraux.

DÉMONSTRATION

```
Pattern : /Dakar/
Texte : "J'habite à Dakar au Sénégal"
Resultat : CORRESPONDANCE TROUVÉE -- la partie soulignée correspond au pattern

Pattern : /Paris/
Texte : "J'habite à Dakar au Sénégal"
Resultat : AUCUNE CORRESPONDANCE -- "Paris" n'est pas dans le texte
```

SENSIBILITÉ À LA CASSE

Par défaut, les regex PHP sont **sensibles à la casse** : `/dakar/` ne correspond PAS à "Dakar" (D majuscule). Pour rendre une regex insensible à la casse, on ajoute le modificateur `i` après le délimiteur fermant : `/dakar/i`. Cela sera détaillé dans la section Modificateurs.

3. Les métacaractères

DÉFINITION

Les **métacaractères** sont des caractères qui ont une signification spéciale dans une regex. Ils ne représentent pas le caractère lui-même, mais une règle ou un comportement. Ce sont eux qui donnent toute la puissance aux expressions régulières.

Le point . : n'importe quel caractère

Le point `.` correspond à **n'importe quel caractère unique**, sauf un saut de ligne.

Pattern	Correspond à	Ne correspond PAS à
<code>/c.t/</code>	"cat", "cut", "c3t", "c-t"...	"ct" (il manque un caractère entre c et t)
<code>/..../</code>	N'importe quelle suite de 4 caractères	Une chaîne de moins de 4 caractères

^ et \$: ancrs de début et fin

`^` signifie "au début de la chaîne" et `$` signifie "à la fin de la chaîne".

Pattern	Signification	Exemple
<code>/^Bonjour/</code>	La chaîne COMMENCE par "Bonjour"	"Bonjour monde" ✔ — "Dis Bonjour" ✖
<code>/monde\$/</code>	La chaîne SE TERMINE par "monde"	"Bonjour monde" ✔ — "monde entier" ✖
<code>/^Bonjour monde\$/</code>	La chaîne est EXACTEMENT "Bonjour monde"	"Bonjour monde" ✔ — "Bonjour monde !" ✖

```
<?php
// Valider qu'un texte commence par une majuscule
preg_match('/^[A-Z]/', "Khadija");    // true
preg_match('/^[A-Z]/', "khadija");    // false

// Valider qu'un texte se termine par un point d'exclamation
preg_match('/!$/', "Bravo !");        // true
preg_match('/!$/', "Bravo.");         // false

?>
```

| : l'alternative (OU logique)

Le pipe `|` signifie "OU". Il permet de chercher l'un ou l'autre motif.

```
<?php
// Trouver "chat" OU "chien" OU "poisson"
preg_match('/chat|chien|poisson/', "J'ai un chien");    // true
preg_match('/chat|chien|poisson/', "J'ai un lapin");    // false

// Valider une couleur parmi une liste
preg_match('/^(rouge|vert|bleu)$/i', "vert");           // true

?>
```

\ : le caractère d'échappement

Le backslash `\` sert à **échapper** un métacaractère pour le traiter comme un caractère littéral. Si on veut chercher un vrai point (et non "n'importe quel caractère"), on écrit `\.`.

Pattern	Signification
<code>/\./</code>	Un vrai point (pas "n'importe quel caractère")
<code>/\\$/</code>	Un vrai symbole dollar (pas "fin de chaîne")
<code>/\^/</code>	Un vrai accent circonflexe (pas "début de chaîne")
<code>/\/</code>	Un vrai slash (si le délimiteur est /)
<code>/\\</code>	Un vrai backslash

4. Les classes de caractères []

DÉFINITION

Une **classe de caractères**, notée entre crochets [], définit un ensemble de caractères parmi lesquels **un seul** doit correspondre. Par exemple [aeiou] correspond à n'importe quelle voyelle.

Syntaxe des classes de caractères

Notation	Signification	Exemple de correspondance
[abc]	Un caractère qui est soit a, soit b, soit c	"a", "b" ou "c"
[a-z]	Une lettre minuscule de a à z	"k", "d", "z"...
[A-Z]	Une lettre majuscule de A à Z	"K", "D", "Z"...
[0-9]	Un chiffre de 0 à 9	"0", "5", "9"...
[a-zA-Z]	N'importe quelle lettre (min ou maj)	"a", "Z", "m"...
[a-zA-Z0-9]	Lettre ou chiffre (alphanumérique)	"a", "Z", "5"...
[^abc]	Tout caractère SAUF a, b ou c (négation avec ^)	"d", "x", "5" mais pas "a", "b", "c"
[^0-9]	Tout caractère qui n'est PAS un chiffre	"a", "!" mais pas "5"

```
<?php
// Valider une plaque d'immatriculation sénégalaise simple (ex: DK-1234-A)
$pattern = '/^[A-Z]{2}-\d{4}-[A-Z]$/';
preg_match($pattern, "DK-1234-A"); // true
preg_match($pattern, "DK-12A4-A"); // false (des lettres dans les chiffres)

// Vérifier qu'une chaîne ne contient QUE des lettres et espaces
$nom = '/^[a-zA-ZÂ-ÿ\s]+$/';
preg_match($nom, "Khadija Diallo"); // true
preg_match($nom, "Khadija123"); // false (contient des chiffres)
?>
```

Classes de caractères prédéfinies (raccourcis)

Pour éviter d'écrire [0-9] ou [a-zA-Z0-9_] à chaque fois, PHP fournit des raccourcis très pratiques appelés **échappements spéciaux** :

Raccourci	Équivalent	Signification
\d	[0-9]	Un digit (chiffre)
\D	[^0-9]	Tout ce qui N'est PAS un chiffre
\w	[a-zA-Z0-9_]	Un caractère de word (lettre, chiffre ou _)
\W	[^a-zA-Z0-9_]	Tout ce qui N'est PAS un caractère de mot
\s	[\t\n\r]	Un caractère d' espace (espace, tabulation, retour ligne)
\S	[^ \t\n\r]	Tout ce qui N'est PAS un espace
\b	(limite)	Limite de mot (entre \w et \W)
\B	(limite)	PAS une limite de mot

ASTUCE MÉMO

Facile à retenir : \d = digit (chiffre), \w = word (mot), \s = space (espace). Et la majuscule donne toujours la négation : \D = PAS un chiffre, \W = PAS un caractère de mot, \S = PAS un espace.

5. Les quantificateurs : *, +, ?, {n,m}

DÉFINITION

Les **quantificateurs** indiquent **combien de fois** le caractère ou groupe qui les précède doit apparaître pour que la correspondance soit trouvée. Ils s'appliquent toujours à l'élément qui les précède immédiatement.

Les 4 quantificateurs de base

Quantificateur	Signification	Exemple	Correspond à
*	0 ou plusieurs fois	<code>/ab*/</code>	"a", "ab", "abb", "abbb"...
+	1 ou plusieurs fois	<code>/ab+/</code>	"ab", "abb", "abbb"... (mais PAS "a" seul)
?	0 ou 1 fois (optionnel)	<code>/colou?r/</code>	"color" ou "colour" (le u est optionnel)
{n}	Exactement n fois	<code>/\d{4}/</code>	Exactement 4 chiffres : "2024"
{n,}	Au moins n fois	<code>/\d{2,}/</code>	2 chiffres minimum : "12", "123", "1234"...
{n,m}	Entre n et m fois (inclus)	<code>/\d{2,4}/</code>	Entre 2 et 4 chiffres : "12", "123", "1234"

```
<?php
// Valider un numéro de téléphone sénégalais (77, 78, 76, 70 + 7 chiffres)
preg_match('/^(77|78|76|70)\d{7}$/',' 776543210'); // true

// Valider un code postal (4 à 6 chiffres)
preg_match('/^\d{4,6}$/',' 12345'); // true
preg_match('/^\d{4,6}$/',' 123'); // false (trop court)

// Valider un mot de passe : minimum 8 caractères
preg_match('/^.{8,}$/',' monpass1'); // true (8 car.)
preg_match('/^.{8,}$/',' abc'); // false (3 car.)

?>
```

Quantificateurs possessifs et paresseux

Par défaut, les quantificateurs sont **gourmands (greedy)** : ils capturent le maximum possible. On peut les rendre **paresseux (lazy)** en ajoutant **?** après eux : ils capturent alors le minimum possible.

Gourmand	Paresseux	Différence
*	<code>*?</code>	Le plus long / le plus court possible
+	<code>+?</code>	Le plus long / le plus court possible
{n,m}	<code>{n,m}?</code>	Préfère n / préfère m

```
<?php
$html = "<b>Bonjour</b> et <b>Bonsoir</b>";

// Gourmand : capture du premier < au DERNIER >
preg_match('/<.+>/', $html, $m);
// $m[0] = "<b>Bonjour</b> et <b>Bonsoir</b>" (tout capturé)

// Paresseux : capture du premier < au PREMIER >
preg_match('/<.+?>/', $html, $m);
// $m[0] = "<b>" (seulement la première balise)

?>
```

6. Groupements et captures ()

DÉFINITION

Les **parenthèses** dans une regex servent à deux choses : (1) **grouper** des éléments pour y appliquer un quantificateur, et (2) **capturer** une partie du texte correspondant pour pouvoir la réutiliser ensuite.

Groupement sans capture : (?:...)

Quand on veut juste grouper sans récupérer le contenu, on utilise `(?:...)`.

```
<?php
// Sans (?:) : le groupe est capturé
preg_match('/^(ha)+$/', "hahaha", $m);
// $m[0]="hahaha"  $m[1]="ha" (dernier groupe capturé)

// Avec (?:) : le groupe n'est pas capturé
preg_match('/^(?:ha)+$/', "hahaha", $m);
// $m[0]="hahaha" (pas de $m[1])

?>
```

Capture et réutilisation des groupes

Chaque groupe entre parenthèses est numéroté de gauche à droite, à partir de 1. PHP les stocke dans le tableau `$matches`.

```
<?php
$date = "26/06/2026";

// 3 groupes capturés : jour, mois, année
preg_match('/^(\\d{2})\\/(\\d{2})\\/(\\d{4})$/', $date, $matches);

echo $matches[0]; // "26/06/2026" (toute la correspondance)
echo $matches[1]; // "26" (groupe 1 : le jour)
echo $matches[2]; // "06" (groupe 2 : le mois)
echo $matches[3]; // "2026" (groupe 3 : l'année)

// Reformater la date en année-mois-jour
$iso = $matches[3] . "-" . $matches[2] . "-" . $matches[1];
echo $iso; // "2026-06-26"

?>
```

Groupes nommés : (?P<nom>...)

On peut donner un nom à chaque groupe pour le retrouver plus facilement, surtout quand il y a beaucoup de groupes.

```
<?php
$email = "khadija@sonatel.sn";
$pattern = '/^(?P<user>[^\@]+)@(?P<domaine>[^\.]+)\.(?P<ext>[a-z]{2,})$/';

preg_match($pattern, $email, $m);

echo $m["user"]; // "khadija"
echo $m["domaine"]; // "sonatel"
echo $m["ext"]; // "sn"

?>
```

7. Ancrages, limites de mots et assertions

Les limites de mots : \b et \B

DÉFINITION

`\b` (word **b**oundary) représente une **limite de mot** : la frontière entre un caractère alphanumérique (`\w`) et un caractère non-alphanumérique (`\W`), ou entre le début/fin de chaîne et un caractère alphanumérique. Cela permet de chercher des mots entiers, pas juste des sous-chaînes.

```
<?php
// Sans \b : trouve "chat" même dans "château" ou "achat"
preg_match('/chat/', "J'ai acheté un château"); // true (trouvé dans "acheté" et "château")

// Avec \b : trouve "chat" seulement comme mot entier
preg_match('/\bchat\b/', "J'ai un chat blanc"); // true
preg_match('/\bchat\b/', "J'ai acheté un château"); // false

// \B : position qui N'est PAS une limite de mot
preg_match('/\Bchat\B/', "cachat"); // true ("chat" est à l'intérieur d'un mot)

?>
```

Les assertions (lookahead et lookbehind)

DÉFINITION

Les **assertions** sont des conditions qui vérifient ce qui se trouve avant ou après un point dans le texte, **sans capturer ces caractères**. On les appelle aussi "zero-width assertions" car elles ne consomment pas de caractères.

Assertion	Nom	Signification
<code>(?=...)</code>	Lookahead positif	Vérifie que ce qui SUIVIT correspond au motif
<code>(?!...)</code>	Lookahead négatif	Vérifie que ce qui SUIVIT NE correspond PAS
<code>(?<=...)</code>	Lookbehind positif	Vérifie que ce qui PRÉCÈDE correspond au motif
<code>(?<!=...)</code>	Lookbehind négatif	Vérifie que ce qui PRÉCÈDE NE correspond PAS

```
<?php
// Lookahead positif : trouver des nombres suivis de "€"
preg_match_all('/\d+(?=€)/', "5€ et 10$ et 20€", $m);
// $m[0] = ["5", "20"] (seulement les montants en euros)

// Lookahead négatif : mots qui ne sont PAS suivis de "s"
preg_match_all('/\w+(?!s)\b/', "chats et chien", $m);

// Lookbehind positif : trouver des nombres précédés de "$"
preg_match_all('/(?<=\$)\d+/', "5€ et 10$ et 20€", $m);
// $m[0] = ["10"] (seulement le montant en dollars)

?>
```


8. Les modificateurs de motif

DÉFINITION

Les **modificateurs** (aussi appelés *flags*) sont des lettres placées après le délimiteur fermant de la regex. Ils **modifient le comportement global** de l'expression régulière sans changer le motif lui-même.

Tableau de tous les modificateurs PHP

Modificateur	Nom	Effet
i	Case Insensitive	Rend la recherche insensible à la casse (majuscules = minuscules).
m	Multiline	^ et \$ correspondent au début/fin de chaque LIGNE, pas juste de la chaîne entière.
s	Dotall (Single line)	Le point . correspond AUSSI aux sauts de ligne \n (par défaut il ne les capture pas).
x	Extended	Ignore les espaces et les commentaires dans la regex (pour mieux la formater sur plusieurs lignes).
u	Unicode	Active le support complet de l'Unicode (pour les caractères accentués, arabes, etc.).
g	Global	N'existe PAS en PHP comme modificateur. Utiliser preg_match_all() à la place.

```
<?php
// i : insensible à la casse
preg_match('/dakar/i', "J'habite à DAKAR"); // true
preg_match('/dakar/i', "J'habite à Dakar"); // true
preg_match('/dakar/i', "J'habite à dakar"); // true

// m : multiline — ^ et $ s'appliquent à chaque ligne
$texte = "Ligne 1\nLigne 2\nLigne 3";
preg_match_all('/^Ligne/m', $texte, $m);
// $m[0] = ["Ligne", "Ligne", "Ligne"] — trouvé 3 fois (une par ligne)

// s : dotall — le point capture aussi les sauts de ligne
$html = "<p>\n  Bonjour\n</p>";
preg_match('/
.*</p>/s', $html, $m); // true (capture le \n)

// x : extended — espaces et commentaires ignorés dans la regex
$pattern = '/
    ^          # début de chaîne
    \d{2}      # deux chiffres (jour)
    \\/       # slash
    \d{2}      # deux chiffres (mois)
    \\/       # slash
    \d{4}      # quatre chiffres (année)
    $          # fin de chaîne
/x';
preg_match($pattern, "26/06/2026"); // true

// u : unicode — pour les accents et caractères spéciaux
preg_match('/^[a-zà-ÿ]+$ /u', "bonjour"); // true avec accents
?>
```

Combiner plusieurs modificateurs

```
<?php
// Insensible à la casse ET multiline
preg_match_all('/^bonjour/im', "Bonjour\nBONJOUR\nSalut", $m);
// Trouve 2 correspondances : "Bonjour" et "BONJOUR"
?>
```

9. preg_match() et preg_match_all()

preg_match() : vérifier si un motif existe

DÉFINITION

`preg_match()` recherche la **première occurrence** d'un motif dans une chaîne. Elle retourne **1** si trouvée, **0** si non trouvée, et **false** en cas d'erreur dans le pattern.

```
<?php
// Syntaxe : preg_match($pattern, $sujet, $matches, $flags, $offset)

// Utilisation simple : juste vérifier si ça correspond
$resultat = preg_match('/\d+/', "J'ai 19 ans");
echo $resultat; // 1 (trouvé)

// Avec $matches : récupérer ce qui a été trouvé
preg_match('/(\d+) ans/', "J'ai 19 ans", $matches);
echo $matches[0]; // "19 ans" (toute la correspondance)
echo $matches[1]; // "19" (groupe capturé 1)

// Validation d'un mot de passe fort :
// - au moins 8 caractères
// - au moins une majuscule
// - au moins un chiffre
// - au moins un caractère spécial
function validerMotDePasse($mdp) {
    return preg_match('/^(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,}$/ ', $mdp);
}
echo validerMotDePasse("Dakar2024!"); // 1 (valide)
echo validerMotDePasse("simple"); // 0 (invalide)

?>
```

preg_match_all() : trouver toutes les occurrences

DÉFINITION

`preg_match_all()` recherche **toutes les occurrences** du motif dans la chaîne et les retourne dans le tableau `$matches`. Elle retourne le **nombre total de correspondances** trouvées.

```
<?php
$texte = "Mon email est khadija@sonatel.sn et aussi moussa@gmail.com";

// Extraire tous les emails du texte
$pattern = '/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/';
$nb = preg_match_all($pattern, $texte, $emails);

echo $nb . " email(s) trouvé(s)"; // 2 email(s) trouvé(s)
print_r($emails[0]);
// Array ( [0] => khadija@sonatel.sn [1] => moussa@gmail.com )

// Extraire tous les nombres d'un texte
$phrase = "Il y a 3 chats, 12 chiens et 1 poisson";
preg_match_all('/\d+/', $phrase, $nombres);
// $nombres[0] = ["3", "12", "1"]

?>
```

10. preg_replace() et preg_replace_callback()

preg_replace() : remplacer selon un motif

DÉFINITION

`preg_replace()` recherche toutes les correspondances d'un motif dans une chaîne et les **remplace par une nouvelle valeur**. Elle retourne la chaîne modifiée.

```
<?php
// Syntaxe : preg_replace($pattern, $remplacement, $sujet)

// Supprimer tous les chiffres d'un texte
$texte = "Khadija23 Diallo45";
$resultat = preg_replace('/\d/', '', $texte);
echo $resultat;    // "Khadija Diallo"

// Remplacer plusieurs espaces consécutifs par un seul
$phrase = "Bonjour    tout    le    monde";
echo preg_replace('/\s+/', ' ', $phrase);
// "Bonjour tout le monde"

// Reformater une date avec les groupes capturés ($1, $2, $3)
$date = "26/06/2026";
$iso = preg_replace('/^(\d{2})\./(\d{2})\./(\d{4})$/ ', '$3-$2-$1', $date);
echo $iso;    // "2026-06-26"

// Masquer les numéros de carte bancaire (garder les 4 derniers chiffres)
$carte = "4532 1234 5678 9012";
$masquee = preg_replace('/\d{4}/', '*', $carte);
echo $masquee;    // "**** * 9012"

?>
```

preg_replace_callback() : remplacement avec une fonction

DÉFINITION

`preg_replace_callback()` est comme `preg_replace()`, mais au lieu d'un texte de remplacement fixe, on fournit une **fonction** qui reçoit chaque correspondance et calcule dynamiquement la valeur de remplacement.

```
<?php
// Mettre en majuscules le premier mot de chaque phrase
$texte = "bonjour monde. comment vas-tu. bien merci.";
$resultat = preg_replace_callback(
    '/(?:^|[.!?]\s+)([a-z])/ ',
    function($matches) {
        return str_replace($matches[1], strtoupper($matches[1]), $matches[0]);
    },
    $texte
);
// "Bonjour monde. Comment vas-tu. Bien merci."

// Ajouter la TVA à tous les prix trouvés dans un texte
$tarifs = "Article A: 1000 FCFA, Article B: 5000 FCFA";
$avecTVA = preg_replace_callback(
    '/(\d+) FCFA/',
    function($m) {
        $ttc = (int)$m[1] * 1.18;
        return $ttc . " FCFA TTC";
    },
    $tarifs
);
// "Article A: 1180 FCFA TTC, Article B: 5900 FCFA TTC"

?>
```

11. preg_split(), preg_grep() et preg_quote()

preg_split() : découper selon un motif

DÉFINITION

`preg_split()` découpe une chaîne en tableau en utilisant un motif regex comme séparateur. C'est bien plus puissant qu' `explode()` qui n'accepte qu'un séparateur fixe.

```
<?php
// explode() : séparateur fixe, moins flexible
$tab1 = explode(",", "Dakar,Thiès,Ziguinchor");
// ["Dakar", "Thiès", "Ziguinchor"]

// preg_split() : séparateur flexible (virgule ou point-virgule ou espace)
$tab2 = preg_split('/[;,\\s]+/', "Dakar, Thiès; Ziguinchor Saint-Louis");
// ["Dakar", "Thiès", "Ziguinchor", "Saint-Louis"]

// Découper un texte en phrases
$texte = "Bonjour. Comment vas-tu ? Bien, merci !";
$phrases = preg_split('/[.?!]\\s*/', $texte, -1, PREG_SPLIT_NO_EMPTY);
// ["Bonjour", "Comment vas-tu", "Bien, merci"]

// Découper en gardant le délimiteur dans les résultats
$mots = preg_split('/(\\s+)/', "Bonjour Khadija", -1, PREG_SPLIT_DELIM_CAPTURE);
// ["Bonjour", " ", "Khadija"]

?>
```

preg_grep() : filtrer un tableau selon un motif

DÉFINITION

`preg_grep()` filtre un tableau et retourne uniquement les éléments qui correspondent au motif. C'est comme `array_filter()` mais avec une regex au lieu d'une fonction.

```
<?php
$villes = ["Dakar", "THIÈS", "Saint-Louis", "ZIGUINCHOR", "Kaolack"];

// Garder seulement les villes entièrement en MAJUSCULES
$majuscules = preg_grep('/^[A-Z\\s-]+$/', $villes);
// ["THIÈS", "ZIGUINCHOR"]

// Garder seulement les emails d'un domaine .sn
$emails = ["k@sonatel.sn", "m@gmail.com", "f@orange.sn"];
$senegalais = preg_grep('/\\.sn$/', $emails);
// ["k@sonatel.sn", "f@orange.sn"]

// Avec PREG_GREP_INVERT : retourner les NON-correspondances
$autres = preg_grep('/\\.sn$/', $emails, PREG_GREP_INVERT);
// ["m@gmail.com"]

?>
```

preg_quote() : échapper les caractères spéciaux

DÉFINITION

`preg_quote()` est essentielle quand on utilise une variable comme motif de recherche. Elle **échappe automatiquement tous les caractères spéciaux** d'une chaîne pour qu'elle soit traitée comme du texte littéral, sans risque qu'un caractère soit interprété comme un métacaractère.

```
<?php
// Sans preg_quote : DANGEREUX si $recherche contient des . ( ) etc.
$recherche = "3.14";
preg_match("/$recherche/", "3X14"); // true par erreur (. = n'importe quel char)

// Avec preg_quote : SÉCURISÉ
$recherche_safe = preg_quote($recherche, "/"); // "3\\.14"
preg_match("/$recherche_safe/", "3X14"); // false (le . est échappé)
```

```
preg_match("/$recherche_safe/", "3.14"); // true (le vrai point)
?>
```

12. Gestion des erreurs et résumé complet

Gestion des erreurs avec preg_last_error()

DÉFINITION

`preg_last_error()` retourne le code d'erreur de la dernière opération regex effectuée. Elle retourne une constante entière qu'on compare aux constantes `PREG_*` pour identifier le type d'erreur.

Constante	Valeur	Signification
<code>PREG_NO_ERROR</code>	0	Pas d'erreur — tout s'est bien passé
<code>PREG_INTERNAL_ERROR</code>	1	Erreur interne du moteur PCRE
<code>PREG_BACKTRACK_LIMIT_ERROR</code>	2	Limite de retour arrière atteinte (pattern trop complexe)
<code>PREG_RECURSION_LIMIT_ERROR</code>	3	Limite de récursion atteinte
<code>PREG_BAD_UTF8_ERROR</code>	4	Chaîne UTF-8 malformée avec le modificateur u
<code>PREG_BAD_UTF8_OFFSET_ERROR</code>	5	Offset invalide dans une chaîne UTF-8
<code>PREG_JIT_STACKLIMIT_ERROR</code>	6	Limite de la pile JIT atteinte

```
<?php
// Vérifier si une regex a généré une erreur
$resultat = preg_match('/pattern/', $sujet);

if ($resultat === false) {
    $erreur = preg_last_error();
    switch ($erreur) {
        case PREG_BACKTRACK_LIMIT_ERROR:
            echo "Pattern trop complexe — limite de retour arrière atteinte";
            break;
        case PREG_BAD_UTF8_ERROR:
            echo "Encodage UTF-8 invalide";
            break;
        default:
            echo "Erreur regex code : " . $erreur;
    }
}

// PHP 8+ : preg_last_error_msg() retourne le message texte directement
echo preg_last_error_msg();
?>
```

Tableau de référence rapide : tous les éléments d'une regex

Élément	Signification
<code>.</code>	N'importe quel caractère (sauf \n)
<code>^</code>	Début de chaîne (ou de ligne avec /m)
<code>\$</code>	Fin de chaîne (ou de ligne avec /m)
<code>*</code>	0 ou plusieurs fois
<code>+</code>	1 ou plusieurs fois
<code>?</code>	0 ou 1 fois (optionnel)
<code>{n,m}</code>	Entre n et m fois
<code>[abc]</code>	Un caractère parmi a, b, c
<code>[^abc]</code>	Tout sauf a, b, c
<code>(abc)</code>	Groupe capturant
<code>(?:abc)</code>	Groupe non-capturant

<code>(?P<nom>)</code>	Groupe nommé
<code>a b</code>	a OU b
<code>\d \D</code>	Chiffre / Non-chiffre
<code>\w \W</code>	Alphanumérique / Non-alphanumérique
<code>\s \S</code>	Espace / Non-espace
<code>\b \B</code>	Limite de mot / Non-limite
<code>(?=...)</code>	Lookahead positif
<code>(?!...)</code>	Lookahead négatif
<code>(?<=...)</code>	Lookbehind positif
<code>(?<!=...)</code>	Lookbehind négatif
<code>i m s x u</code>	Modificateurs (insensible / multiline / dotall / extended / unicode)

🕒 **La phrase à retenir pour ta présentation**

"Une expression régulière est un motif qui décrit le format d'un texte. En PHP, on les écrit entre délimiteurs `/.../` et on les utilise avec les fonctions `preg_*` : `preg_match()` pour vérifier, `preg_match_all()` pour tout trouver, `preg_replace()` pour remplacer, `preg_split()` pour découper, `preg_grep()` pour filtrer un tableau, et `preg_quote()` pour sécuriser une variable utilisée comme motif."